二回にわたって確かめた —— パラメータリストのケイパビリティがエフェクトを可視にすること、 そしてそれが高階関数を生き延びること。だが、どの解像度で可視なのか？ ある関数が Database を手渡されたと知っても、それはデータベースに触れられることを告げるだけだ —— ただ読むのか書きもするのかも、データベースを独り占めしたいのか共有できるのかも告げない。 有無は粗い粒度だ。今回は、エフェクトにより細かい粒度を与える話であり、それをどう与えるかを 決める一つの厄介なケースの話だ。

Logger 問題

二点しかないシステムが表現できないケースがこれだ：

```
fn read_users(db: &borrowed Database) -> Vec[User]       // 読み
fn save_order(db: &mut borrowed Database) -> unit        // 書き、排他
fn log_info(logger: &borrowed Logger, msg: &borrowed str)  // 書き —— だが共有可能
```

最初の二つは居心地がよい。ユーザを読むには共有の読み取り専用ハンドルが要り、注文を保存する には排他的な書き込みアクセスが要る。それらは、たいていの言語がくれる二つの借用形に綺麗に 対応する：共有読みの & と、排他書きの &mut だ。

三つ目が対応を壊す。logger は書き込まれる —— ログは行を追記する —— から、読みではない。 だが排他的でありたくもない：logger の要点はまさに、多くの呼び手が、多くのスレッドさえもが、 同時に書き込むことで、安全性は内部で（ロック、あるいはロックフリーのキュー）担われる。それを &borrowed

Rust が結びつける二つの軸

言葉がない理由は、& と &mut がそれぞれ一度に二つの仕事をしているからだ。& は共有かつ 読み取り専用を意味し、&mut は排他かつ書き込み可を意味する。二つの独立した問い —— これを複数の借用者が同時に持てるか？ そして借用者はそれを通して変更できるか？ —— が、一つの 二者択一に融合されている。その融合は便利で、大半のコードを覆うが、四つの組み合わせのうち一つを名づけられないまま残す：共有かつ書き込み可。それがまさに logger だ。

Mere の応答は、連載が何度も見せてきた習慣から直に従う：一つの名前がこっそり二つの意味を運んで いるなら、割る。だから借用注釈は二つの直交軸のグリッドになる —— 共有性（shared / exclusive） と操作（read / write）—— そして四点それぞれに名がつく：

注釈	共有性	書き	例

そもそもなぜ型に載せるのか

より安い答えもあった。検討された代替は、「単一の & を保ち、何が許されるかは各ケイパビリティ の API に決めさせる」から、Logger<Write, Shared> のようなマーカータグをケイパビリティ型に 付けるまでに及んだ。単一借用の選択肢は建てるのが最も簡単だ —— だがそれは、並行性の問いを丸ごと ケイパビリティの実装へ押しやり、そこは型システムに見えない：共有ハンドル越しの cache.set が 安全かどうかは、コンパイラが検査する事実ではなく、作者がする約束になる。検証できるものの 最大化を前提の全てとする言語にとって、並行安全性を未検査の慣習に委ねるのは誤った取引だ。だから その選択肢は端的に棄却された。

マーカータグの手法は、グリッドが表現できることの全てとそれ以上を表現できる —— Io タグも、 Gpu タグも、何でも発明できる —— が、その一般性こそが

実行時コスト：ゼロ

絞り込みは紙上でなされ、それからいくつかの実装スライスとしてコンパイラに落とされた —— そして 最も物語るのは、何が変わらなくて済んだかだ。四つのモードは & の後の文脈依存キーワードで、 型の中を borrow_mode として乗っていき、単一化はそれらを subtyping なしに区別する。だから &mut R Database が欲しいところに &R Database を渡すのは、呼び出し箇所で捕まる型エラーだ。 だが実行時と三つの backend（C・LLVM・Wasm）はモードをただ無視する：借用はモードによらず ポインタだから、生成されるコードは同一で、backend 間の feature parity はコード生成に一切 触れずに保たれた。これは view 型が Trivial であることと同じ形だ：完全に型システムの中に住み、実行時には 蒸発する保証。&shared write と &mut の区別は、プログラマにシグネチャでのいくらかの正直さを 要求し、走るプログラムには何も要求し

並行処理の章へ先送りしたもの

&shared write に名をつけることは、それがまだ答えない問いを立てる：そう借用されるために、 型は何を証明せねばならないのか？ 共有書き込み可能なケイパビリティは並行アクセスに対して内部 的に安全でなければならず、型がそうだと宣言できる道があるべきだ —— Send/Sync の精神の制約。 そして並行 backend への正確なコード生成 —— ロックが実際どこに入るか、書き込み順序がどう保証 されるか —— は、生成すべき並行 backend ができるまで残される。どちらも意図して先送りされる。 並行処理が、予期されるものではなく現実になる、物語のその Part へ。

これでエフェクトモデルは、関数が何に触れてよいかだけでなくどう —— 読みか書きか、単独か 共有か —— を言えるようになった。残るのは、最初のエフェクト回が旗を立てた人間工学の問題だ： 現実的な関数はいくつものケイパビリ